



UPLA

UNIVERSIDAD PERUANA LOS ANDES

ARQUITECTURA DE BASES DE DATOS Y SU APLICABILIDAD TECNOLÓGICA

Diseño de Arquitectura, Modelado y
Tecnologías de Bases de Datos Relacionales
y No Relacionales.



ESTUDIANTE:

Vera Madueño Jordan

DOCENTE;

Raul Enrique Fernandez Bejarano



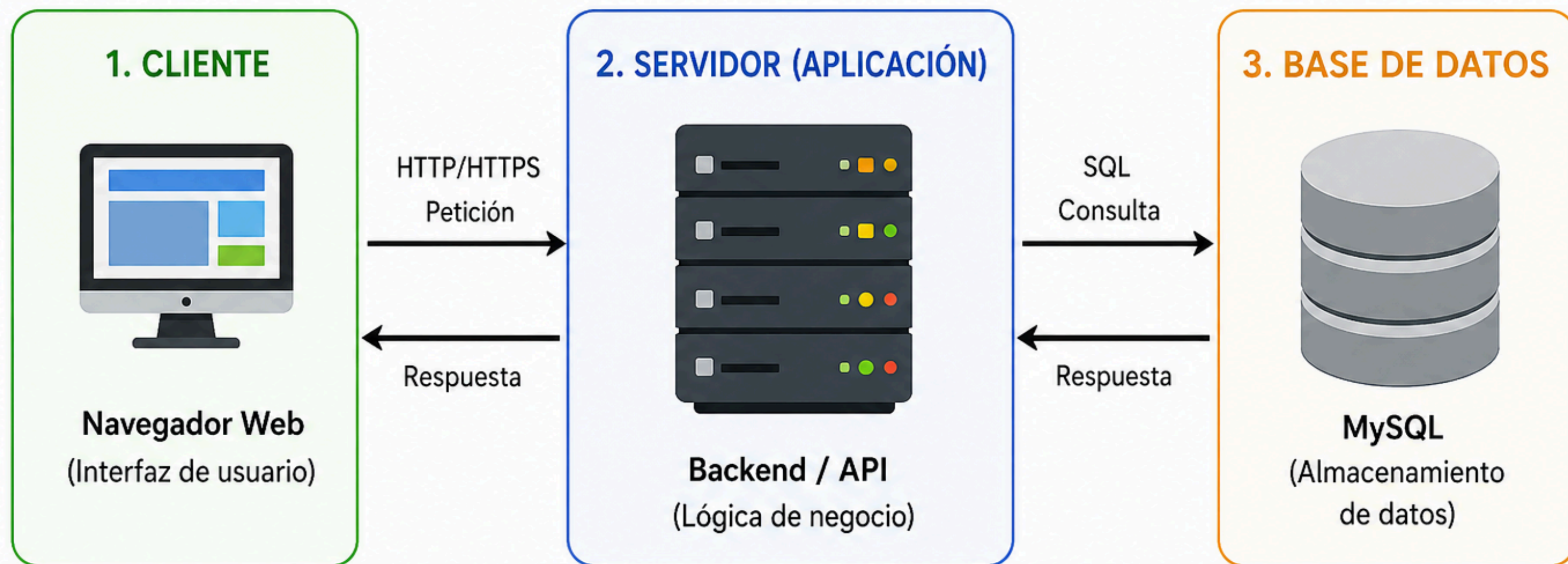
CURSO:

Base de Datos II



EVIDENCIA 1: DIAGRAMA DE ARQUITECTURA

Tipo de arquitectura: Cliente-Servidor (3 capas)



DESCRIPCIÓN

El cliente (navegador) realiza peticiones al servidor de aplicación a través de HTTP/HTTPS. El servidor procesa la lógica de negocio y consulta la base de datos MySQL para almacenar o recuperar información, devolviendo la respuesta al cliente.

EVIDENCIA 2: DOCUMENTO DE DISEÑO LÓGICO

1 Definición de la arquitectura seleccionada

Se seleccionó una arquitectura Cliente-Servidor de tres capas porque permite separar las responsabilidades en presentación, lógica de negocio y almacenamiento de datos, facilitando el mantenimiento, la escalabilidad y la seguridad del sistema.



2 Componentes fundamentales explicados



Cliente: Interfaz de usuario accesible desde un navegador web. Permite al usuario interactuar con el sistema.



Servidor de Aplicación: Procesa la lógica del negocio, valida datos y gestiona las peticiones del cliente.



Base de Datos (MySQL): Almacena la información de manera persistente y segura.



Comunicación: HTTP/HTTPS entre cliente y servidor; SQL entre servidor y base de datos.

3 Justificación en relación con los requisitos del sistema



El sistema requiere accesibilidad multiusuario, integridad de datos, seguridad y facilidad de mantenimiento. La arquitectura cliente-servidor cumple con estos requisitos al centralizar los datos y la lógica, permitir acceso concurrente y facilitar futuras ampliaciones.



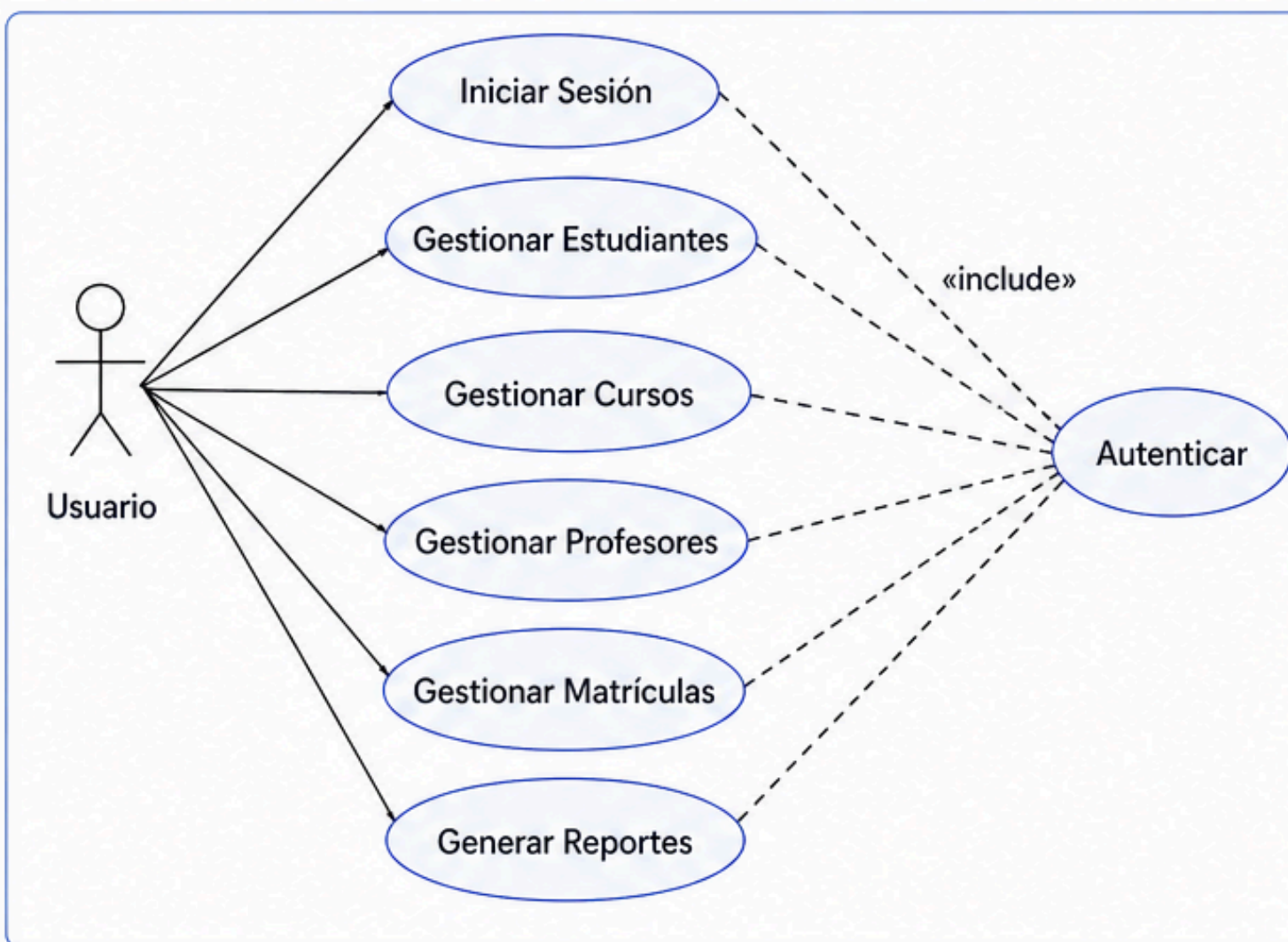
Resumen

La arquitectura propuesta separa claramente las funciones del sistema en tres capas, utilizando tecnologías estándar (HTTP/HTTPS, SQL, MySQL) para garantizar un sistema eficiente, seguro y escalable.

4 Diagramas ER o UML según el caso

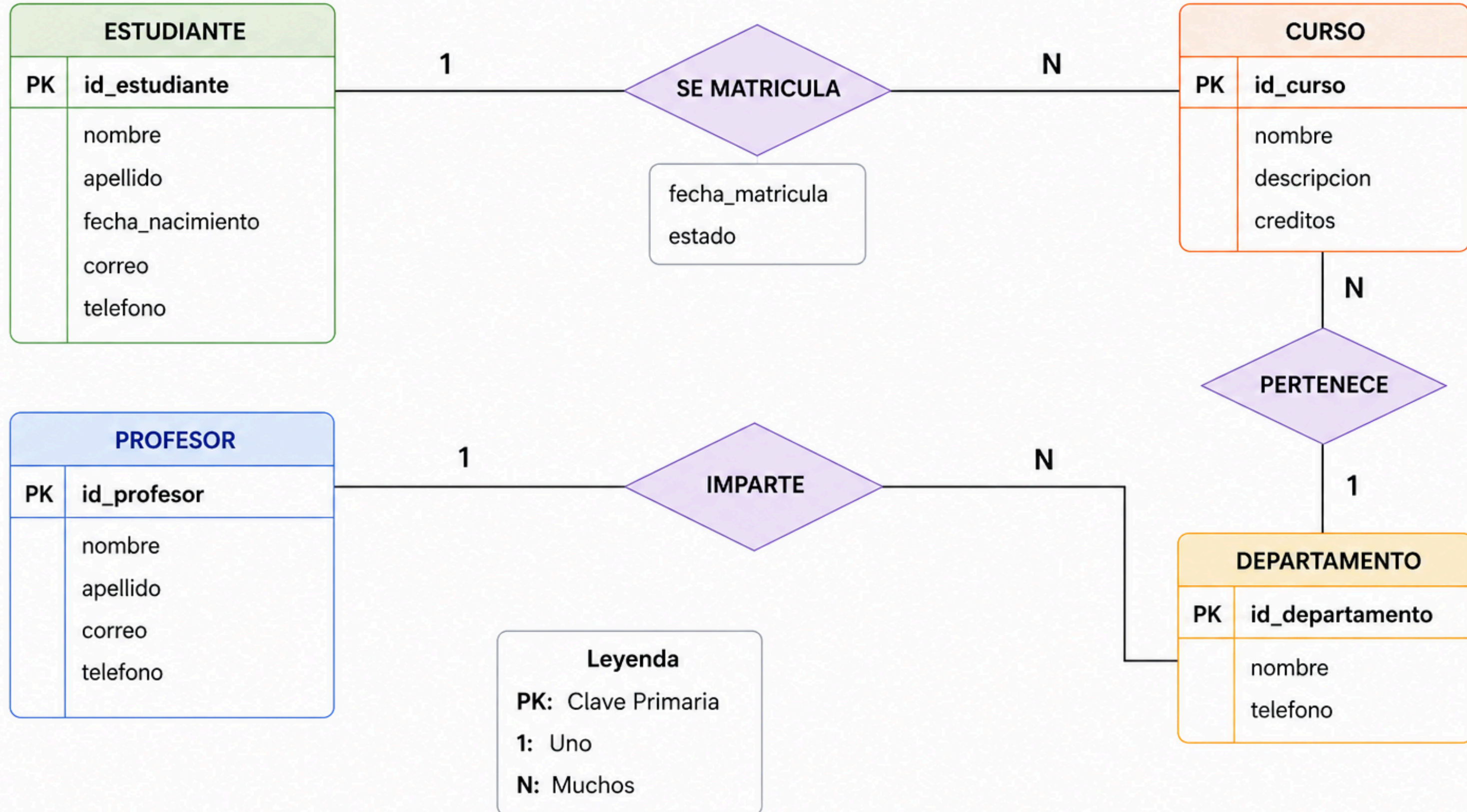
Se presenta el diagrama Entidad-Relación del sistema en la Evidencia 3 y el diagrama de casos de uso UML.

Diagrama de Casos de Uso (UML)



EVIDENCIA 3: DIAGRAMA DE ARQUITECTURA DE BASE DE DATOS

Sistema: Universidad Diagrama Entidad-Relación (ER)



EVIDENCIA 4: DOCUMENTO DE DISEÑO LÓGICO

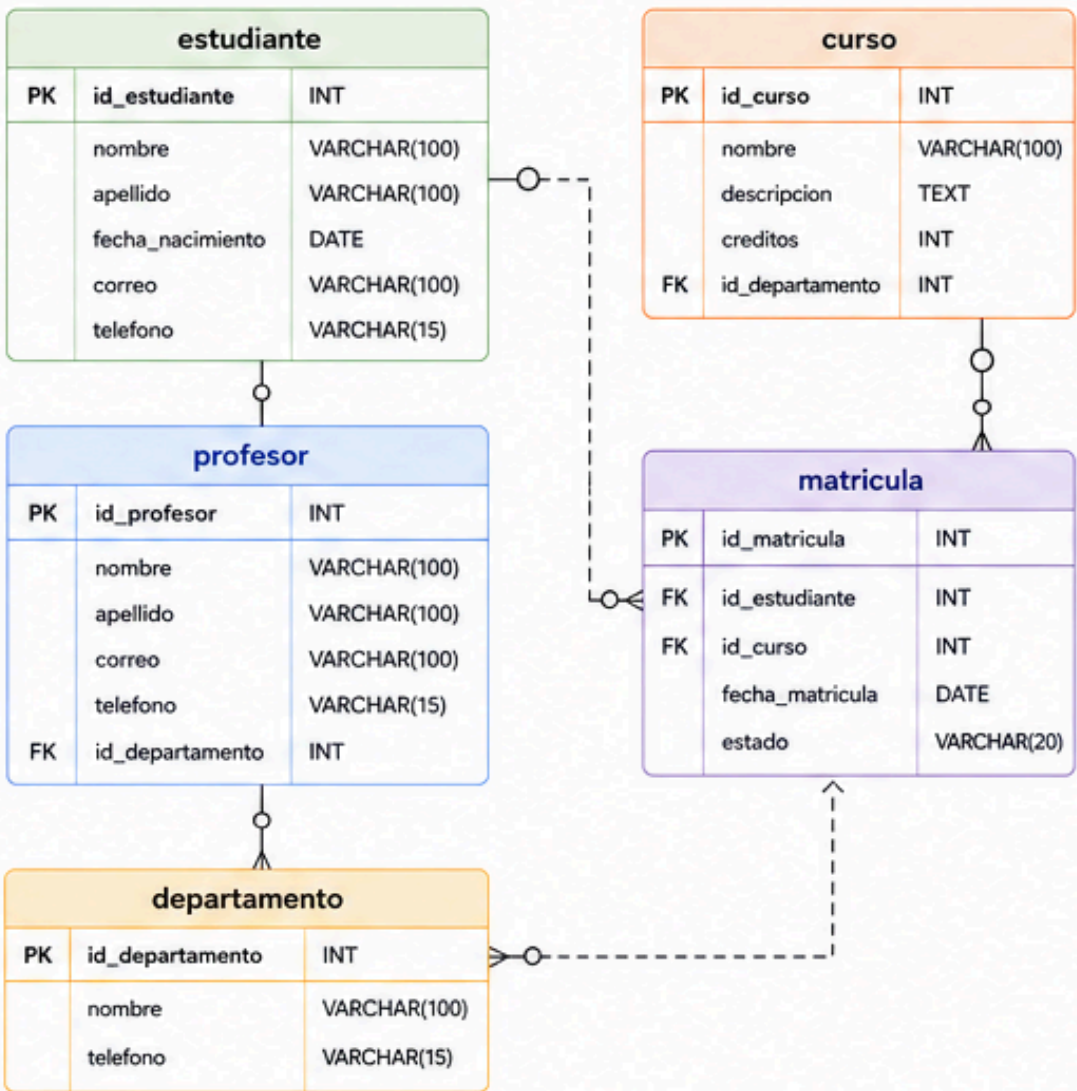


1. MODELO CONCEPTUAL (Idea general del sistema)

El sistema de gestión universitaria permite administrar información de estudiantes, profesores, cursos, departamentos y matrículas. Su objetivo es registrar, consultar y reportar las actividades académicas.



2. MODELO LÓGICO (Tablas y relaciones)



Leyenda
PK: Clave Primaria FK: Clave Foránea
----- : Relación Uno a Muchos
———< : Relación Muchos



3. MODELO FÍSICO (Estructura en la BD)

```
CREATE TABLE departamento (  
  id_departamento INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  telefono VARCHAR(15)  
);
```

```
CREATE TABLE curso (  
  id_curso INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  descripcion TEXT,  
  credits INT NOT NULL,  
  id_departamento INT,  
  FOREIGN KEY (id_departamento)  
    REFERENCES departamento(id_departamento)  
);
```

```
CREATE TABLE profesor (  
  id_profesor INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  apellido VARCHAR(100) NOT NULL,  
  correo VARCHAR(100),  
  telefono VARCHAR(15),  
  id_departamento INT,  
  FOREIGN KEY (id_departamento)  
    REFERENCES departamento(id_departamento)  
);
```

```
CREATE TABLE estudiante (  
  id_estudiante INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  apellido VARCHAR(100) NOT NULL,  
  fecha_nacimiento DATE,  
  correo VARCHAR(100),  
  telefono VARCHAR(15)  
);
```

```
CREATE TABLE matricula (  
  id_matricula INT PRIMARY KEY,  
  id_estudiante INT,  
  id_curso INT,  
  fecha_matricula DATE NOT NULL,  
  estado VARCHAR(20),  
  FOREIGN KEY (id_estudiante)  
    REFERENCES estudiante(id_estudiante),  
  FOREIGN KEY (id_curso)  
    REFERENCES curso(id_curso)  
);
```



4. TÉCNICAS DE MODELADO APLICADAS

- Identificación de entidades y atributos.
- Definición de relaciones y cardinalidades.
- Normalización de datos hasta 3FN.



5. JUSTIFICACIÓN DE NORMALIZACIÓN / DESNORMALIZACIÓN

La base de datos está en Tercera Forma Normal (3FN) para evitar redundancia y garantizar integridad de los datos.

No se aplica desnormalización en este diseño, ya que el volumen de datos no requiere optimizaciones adicionales.



6. HERRAMIENTAS UTILIZADAS

- Draw.io (diagramas de arquitectura y ER)
- MySQL Workbench (modelo lógico y físico)
- Microsoft Word (documentación)



7. CONCLUSIÓN

El diseño lógico propuesto cumple con los requisitos del sistema universitario, asegurando integridad, consistencia y escalabilidad. La arquitectura cliente-servidor y el modelo de base de datos relacional permiten un sistema eficiente, seguro y fácil de mantener.